



Московский государственный университет имени
М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра автоматизации систем вычислительных комплексов

Рогинко Алла Дмитриевна

**Исследование и реализация жадных
алгоритмов
для задачи распределения таблиц
классификации
по стадиям конвейера СПУ**

Курсовая работа

Научный руководитель:
Доцент, к.ф.-м.н.
Капитонова Алла Петровна
Научный консультант:
Никифоров Никита Игоревич

Москва, 2026

Аннотация

Исследование и реализация жадных алгоритмов
для задачи распределения таблиц классификации
по стадиям конвейера СПУ

Рогинко Алла Дмитриевна

Краткое описание задачи и основных результатов, мотивирующее прочи-
тать весь текст

Abstract

Содержание

1	Введение	4
2	Предметная область	5
2.1	Сетевые процессорные устройства	5
2.2	Архитектура RMT	5
2.3	Программа обработки пакетов и язык P4	6
2.4	Граф зависимостей и виды зависимостей внутри таблиц P4	6
3	Цель работы и постановка задачи	8
3.1	Цель работы	8
3.2	Поставленные задачи:	8
4	Обзор существующих алгоритмов	9
5	Реализованные алгоритмы	10
6	Описание архитектуры реализованных алгоритмов	11
7	Экспериментальное исследование	12
8	Заключение	13
	Список литературы	14

1 Введение

В современных сетях возникает необходимость в гибкой обработке пакетов. Необходимо быстро внедрять новые протоколы и сервисы без замены оборудования. Решением этого вопроса является использование программируемых сетевых устройств с архитектурой RMT с языком описания пакетной обработки P4. Такие устройства реализуют конвейерную обработку, состоящую из последовательных match-action стадий.

При компиляции P4-программы для архитектуры RMT нужно распределить логические таблицы по стадиям конвейера. От этого решения зависят число задействованных стадий и, следовательно, задержка обработки пакета. Для успешного размещения следует учитывать ограничения архитектуры аппаратуры и порядок применения таблиц, заданный графом зависимостей (Table Dependency Graph). На практике стремятся минимизировать число занятых стадий, поскольку оно непосредственно определяет задержку обработки пакета.

В данной работе рассматриваются жадные алгоритмы решения задачи размещения, анализируется их эффективность и быстрота выполнения, и предлагается модификация с поддержкой горизонтального разрезания таблиц.

2 Предметная область

2.1 Сетевые процессорные устройства

Сетевые процессорные устройства (СПУ) — это специализированные системы, предназначенные для обработки сетевого трафика на высоких скоростях. Проще говоря, такие устройства должны очень быстро принимать пакеты, анализировать их и принимать решение о дальнейшей обработке. В отличие от традиционных сетевых устройств, где логика работы часто жёстко зафиксирована на уровне аппаратуры, программируемые СПУ позволяют изменять алгоритмы обработки пакетов и адаптировать их под разные задачи. Отличительной чертой современных СПУ является наличие программируемой логики обработки пакетов, что позволяет адаптировать устройство под новые протоколы и сетевые функции без замены аппаратного обеспечения.

2.2 Архитектура RMT

Одной из архитектур эталонной для построения высокопроизводительных программируемых сетевых устройств является RMT (Reconfigurable Match Tables). Основная идея этой архитектуры заключается в использовании набора настраиваемых таблиц сопоставления (match tables), которые применяются на разных стадиях конвейера обработки пакетов. На каждой стадии на основе выбранного ключа выполняется поиск в одной или нескольких таблицах, где правила сопоставления определяют, какие действия должны быть выполнены над пакетом. Эти действия могут включать изменение заголовков, выбор следующей стадии обработки или передачу пакета на выходной интерфейс. Благодаря такой структуре архитектура остаётся гибкой и позволяет реализовывать различные алгоритмы обработки сетевого трафика.

Match-action стадии — последовательность одинаковых стадий. Каждая стадия содержит:

1. Таблицы точного поиска, реализованные на SRAM.
2. Таблицы тернарного и префиксного поиска, реализованные на TCAM.

3. Кроссбары для подачи произвольных полей из PNV на вход таблиц.
4. Action-блоки для параллельного изменения полей PNV.

2.3 Программа обработки пакетов и язык P4

Для описания поведения программируемого коммутатора используется язык P4 (Programming Protocol-independent Packet Processors). Программа на P4 определяет набор логических match-action таблиц, каждая из которых включает набор полей заголовка, по которым производится поиск и список возможных действий для них. P4 является аппаратно-независимым языком: одна и та же программа может компилироваться для различных устройств.

2.4 Граф зависимостей и виды зависимостей внутри таблиц P4

При компиляции P4-программы в конвейер RMT строится граф зависимостей таблиц (Table Dependency Graph, TDG). Вершинами графа являются логические таблицы, а рёбрами — зависимости между таблицами. Корректное размещение таблиц по стадиям конвейера должно удовлетворять всем видам зависимостей, иначе обработка пакетов будет нарушена. Выделяют четыре основных типа зависимостей:

Match-зависимость возникает, когда таблица A изменяет поле, а таблица B выполняет поиск по этому полю. Эта зависимость соответствует классической зависимости «чтение после записи». Для корректной работы необходимо, чтобы A полностью завершила выполнение своих действий до того, как B начнёт поиск. Поэтому, таблицы с match-зависимостью не могут располагаться в одной стадии конвейера.

Action-зависимость появляется, когда две таблицы A и B изменяют одно и то же поле, и важен результат, полученный в таблице B . При размещении допускается нахождение A и B в одной стадии.

Successor-зависимость заключается в следующем: выполнение таблицы B зависит от результата таблицы A . Таблицы с successor-зависимостью могут размещаться в одной стадии.

Обратная match-зависимость возникает, когда таблица A выполняет поиск по полю, которое впоследствии изменяется таблицей B . Также требует строгого разделения стадий: A должна быть размещена в стадии, строго меньшей, чем B .

3 Цель работы и постановка задачи

3.1 Цель работы

Целью данной работы является исследование жадных алгоритмов решения задачи размещения логических таблиц классификации по стадиям конвейера RMT-коммутатора, а также разработка программной реализации этих алгоритмов для оценки ресурсных затрат Р4-программ.

3.2 Поставленные задачи:

1. Провести анализ существующих жадных эвристик (FFD, FFL, FFLS, MCF) и их модификаций.
2. Разработать многопроходный алгоритм FFD с поддержкой горизонтального разрезания таблиц.
3. Реализовать алгоритм FFLS (First Fit by Level and Size) как эталон для сравнения.
4. Реализовать поддержку учёта всех четырёх типов зависимостей между таблицами (match, action, successor, reverse match).
5. Провести экспериментальное исследование на синтетических конфигурациях, различающихся количеством таблиц и числом записей.
6. Сравнить алгоритмы по метрикам времени выполнения и количества использованных стадий.
7. Проанализировать полученные результаты и сформулировать рекомендации по выбору алгоритма в зависимости от характеристик входной программы.

4 Обзор существующих алгоритмов

Здесь надо декомпозировать большую задачу из постановки на подзадачи и продолжать этот процесс, пока подзадачи не станут достаточно простыми, чтобы их можно было бы решить напрямую (например, поставив какой-то эксперимент или доказав теорему) или найти готовое решение.

5 Реализованные алгоритмы

Если в рамках работы писался какой-то код, здесь должно быть его описание: выбранный язык и библиотеки и мотивы выбора, архитектура, схема функционирования, теоретическая сложность алгоритма, характеристики функционирования (скорость/память).

6 Описание архитектуры реализованных алгоритмов

Здесь надо перечислить все результаты, полученные в ходе работы. Из текста должно быть понятно, в какой мере решена поставленная задача.

7 Экспериментальное исследование

Здесь надо перечислить все результаты, полученные в ходе работы. Из текста должно быть понятно, в какой мере решена поставленная задача.

8 Заключение

Здесь надо перечислить все результаты, полученные в ходе работы. Из текста должно быть понятно, в какой мере решена поставленная задача.

Список литературы

- [1] P4 Language Specification. <https://p4.org/specs/>.